

Understanding CI and CD: The Two Pillars of the DevOps Culture

This article is about understanding continuous integration and continuous delivery, which are pillars of the DevOps culture. Building quality software that meets the expectations of the client requires, among other things, constant monitoring and improvement in the code. This can be automated very efficiently through DevOps practices.



It is important to understand that continuous integration (CI) and continuous delivery (CD) come under the umbrella of the DevOps culture, which brings about effective communication and collaboration between the development and operations teams. Multiple continuous practices are implemented to adopt the DevOps culture. CI and CD are two of them. Continuous testing, continuous deployment, continuous monitoring, continuous notifications, Infrastructure as a Code and configuration management are some other practices organisations follow.

Continuous code inspection

Continuous code inspection helps to improve the quality of code by checking the code against pre-defined rules. There are many open source and commercial tools available in the market for static and dynamic code analysis. As an example, in Figure 1, SonarQube is integrated with one of the most popular automation servers – Jenkins.

Given below is the console log of SonarQube execution:

```
Started by user admin
Building in workspace F:\1.DevOps\2018\JenkinsHomeTemplate\
sharedspace
No credentials specified
> git.exe rev-parse --is-inside-work-tree # timeout=10
Fetching changes from the remote Git repository
> git.exe config remote.origin.url https://github.com/
mitesh51/spring-petclinic.git # timeout=10
Fetching upstream changes from https://github.com/mitesh51/
spring-petclinic.git
> git.exe --version # timeout=10
> git.exe fetch --tags --force --progress https://github.
com/mitesh51/spring-petclinic.git +refs/heads/*:refs/remotes/
origin/*
> git.exe rev-parse "refs/remotes/origin/master^{commit}" #
timeout=10
> git.exe rev-parse "refs/remotes/origin/origin/
master^{commit}" # timeout=10
Checking out Revision
```